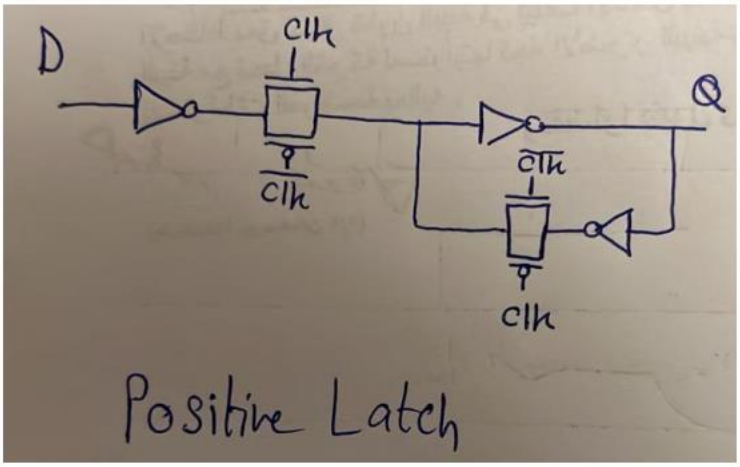
Synchronous Pipelines

Static Latch

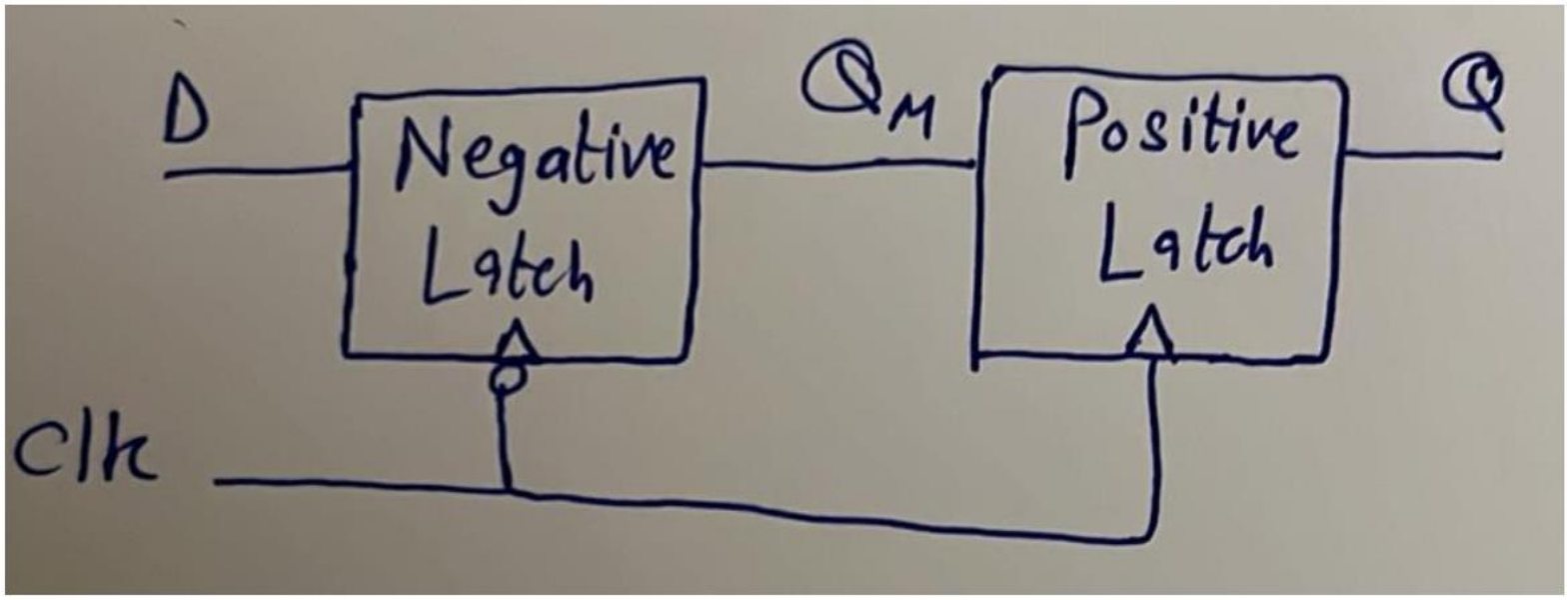


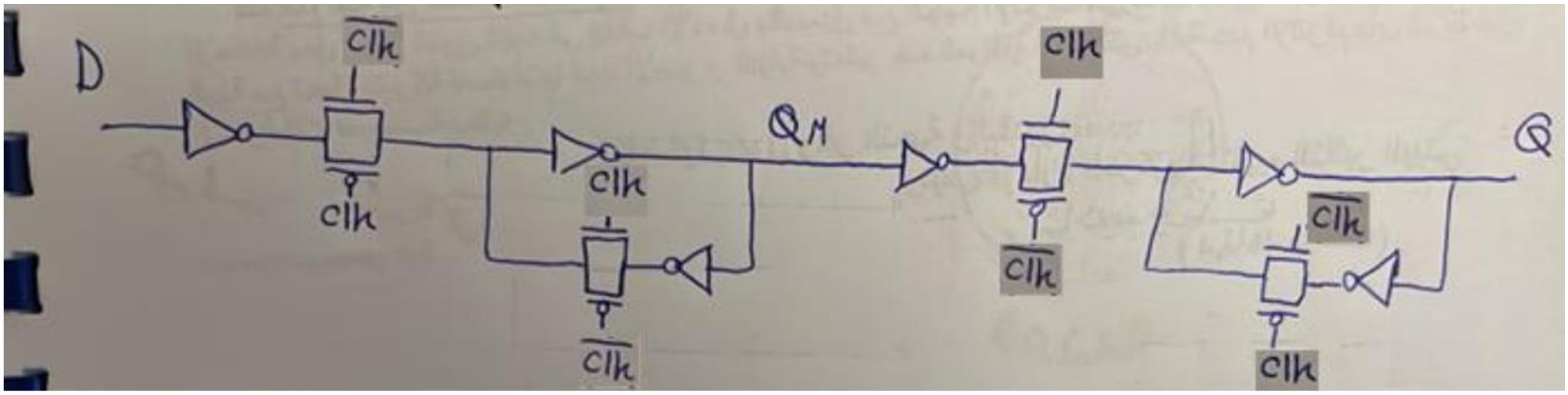
Flip Flops

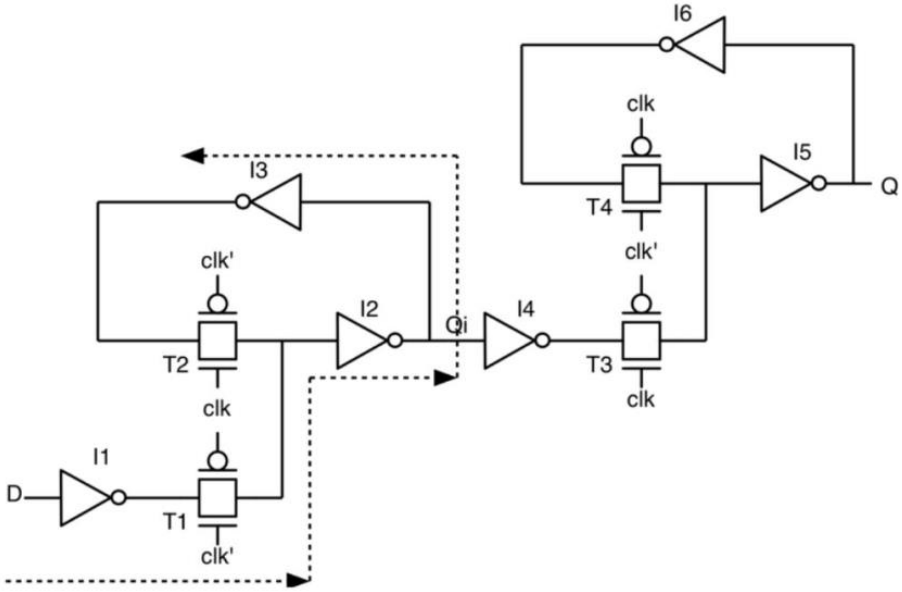
simply, flip-flop is nothing but a negative latch + positive latch. That is:

positive flip-flop = negative latch + positive latch

as shown in the figure below.







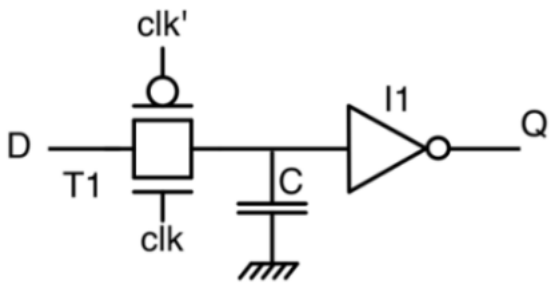
$$t_{su} = t_{I1} + t_{T1} + t_{I2} + t_{I3}$$

$$t_{cq} = t_{T3} + t_{I5}$$

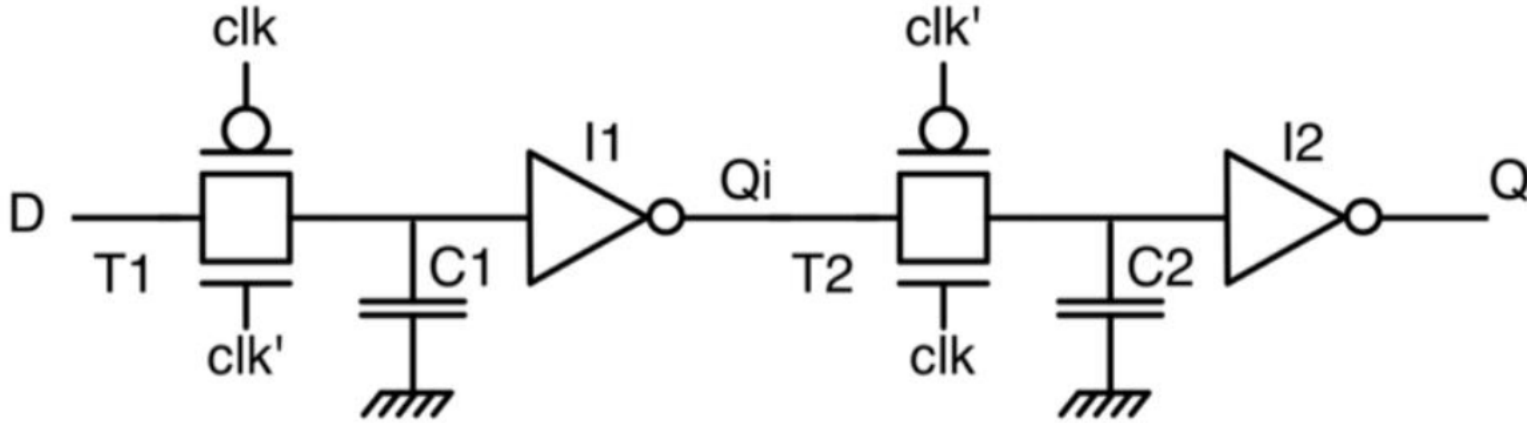
$$t_h = t_{1-1\ overlap} - (t_{I1} + t_{T1})$$

$$t_{0-0\ overlap} < t_{I1} + t_{T1} + t_{I2} + t_{I4} + t_{T3}$$

Dynamic Latch



Dynamic Flip Flop



$$t_{su} = t_{T1}$$

$$t_{cq} = t_{I1} + t_{T2} + t_{I2}$$

$$t_h = t_{1-1\ overlap} - t_{T1}$$

$$t_{0-0\ overlap} < t_{T1} + t_{I1} + t_{T2}$$

- Critical-path delay howa akbar delay maben 2 flip flops. That is, do not include t_{su} and t_{cq} in the critical-path delay, just the longest combinational delay.
- The critical path has zero slack.
- Steps to increase throughput through pipelining:
    1. Register internal nodes in the critical path.
    2. Align all inputs by further registering them.
    3. Align all outputs by further registering them.

Memories

I. The two most important parameters in any memory are density and speed.

II. Elmore steps:
    1. Define the input and output of your circuit. Notice that input and output here doesn't mean the actual inputs and outputs, the output means the node that is to be charged/discharged, and the input means the node that will charge/discharge the output.
    2. The overall time constant is the summation of the time constant of each capacitor in the circuit. But how can we calculate the time constant of a capacitor? betersem path men el input lel output w path men el input lel capacitor da, el resistance el moshtaraka been el 2 paths heya de el resistance el hayshofha el cap.

NOR ROM

1. '1' --> No transistor.
2. '0' --> transistor.
3. All WL = '0' during pre-charging.
4. Delays:

$$t_{\text{read}} = t_{\text{pre}} + t_{\text{wl}} + t_{\text{cell}}$$

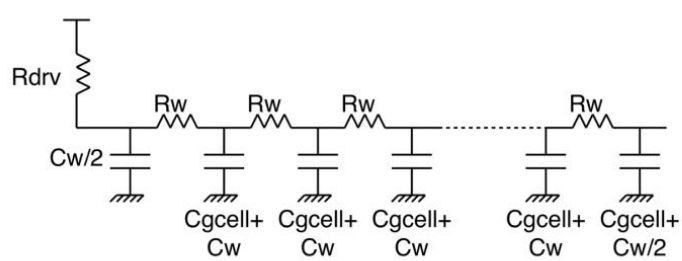
$$\tau_{pre} = R_{pre} C_{BL}$$

$$\tau_{pre} = \frac{R_{p0}}{(W/L)_p} K(C_m + C_{dcell})$$

$$\tau_{cell} = R_{cell} C_{BL}$$

$$\tau_{cell} = \frac{R_{n0}}{(W/L)_{cell}} (C_m + C_{dcell}) K$$

EXACT WORD LINE DELAY CALCULATION



➤ This is calculated using Elmore delay method

$$\tau_{wl} = R_{drv}\frac{C_w}{2} + (R_{drv}+R_w)(C_{gcell}+C_w) + (R_{drv}+2R_w)(C_{gcell}+C_w) + \dots + (R_{drv}+NR_w)(C_{gcell}+\frac{C_w}{2})$$

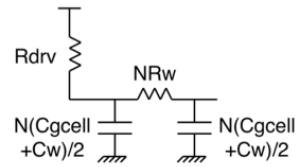
$$= R_{drv}\frac{C_w}{2} + R_{drv}(C_{gcell}+C_w)(N-1) + R_w(C_{gcell}+C_w)\underbrace{(1+2+3+\dots+N-1)}_{\text{arithmetic series}} + (R_{drv}+NR_w)(C_{gcell}+\frac{C_w}{2})$$

Remember: $\sum_{i=1}^{N} a_i = \frac{N}{2}(a_1+a_N)$ $\therefore$ $(1+2+\dots+N-1) = \frac{N-1}{2}(1+N-1) = \frac{N(N-1)}{2}$

$$\therefore \tau_{wl} = N R_{drv}(C_{gcell}+C_w) + \frac{N(N-1)}{2} R_w(C_{gcell}+C_w) + N R_w(C_{gcell}+\frac{C_w}{2})$$

VERY APPROXIMATE WORD LINE DELAY

- ~~Lump all gate capacitances in with word line capacitance~~
  - ~~Use only a single pi section~~
- ~~Valid if the number of columns is very large~~



$$\tau_{wl} = R_{drv}(\frac{NC_w}{2} + \frac{NC_{gcell}}{2}) + (R_{drv} + NR_w)(\frac{NC_w}{2} + \frac{NC_{gcell}}{2})$$

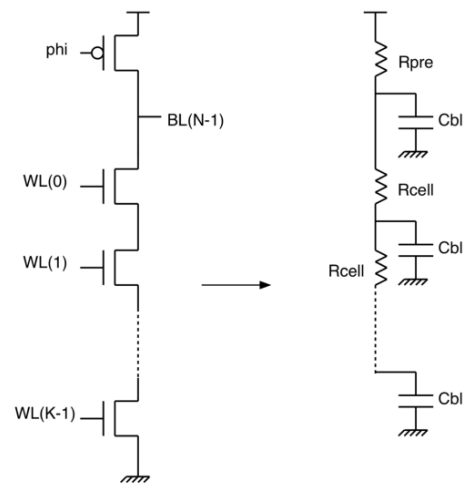
$$\tau_{wl} = NR_{drv}(C_w + C_{gcell}) + \frac{N^2}{2}R_w(C_w + C_{gcell})$$

NAND ROM

1. '0' --> No transistor.
2. '1' --> transistor.
3. All WL = '1' during pre-charging EXCEPT LAST ONE.
4. Delays:

PRECHARGING

- A long RC ladder through the entire bit line
- Assume a transistor in each cel
- Calculate time constant at BL, but taking into consideration loading down to the bottom ce



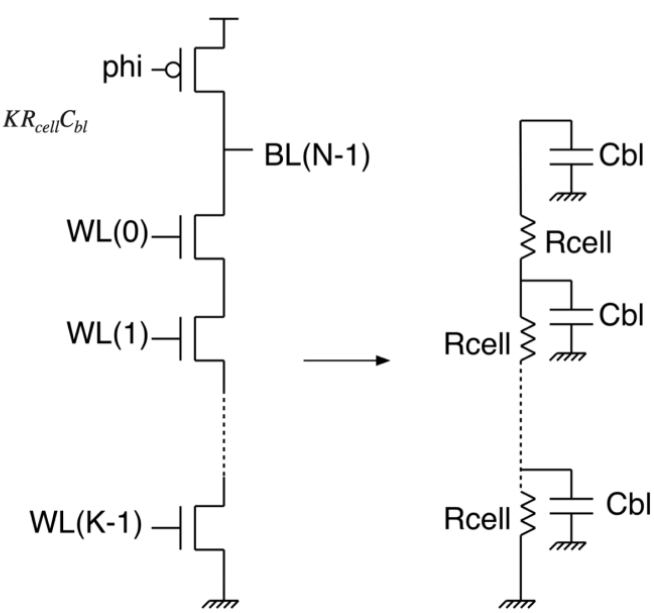
$$\tau_{pre} = R_{pre}C_{bl} + (R_{pre} + R_{cell})C_{bl} + (R_{pre} + 2R_{cell})C_{bl} + \ldots + (R_{pre} + (K-1)R_{cell})C_{bl}$$

CALCULATING CELL DELAY

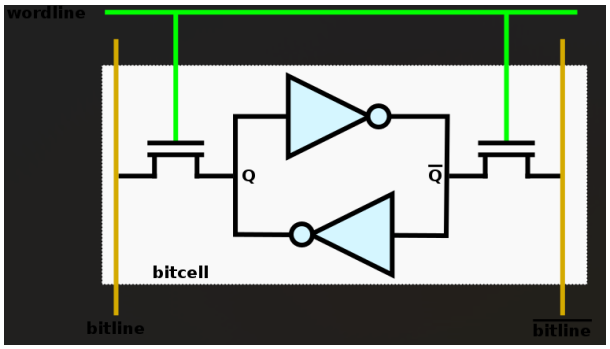
$$\tau_{cell} = R_{cell}C_{bl} + 2R_{cell}C_{bl} + 3R_{cell}C_{bl} + \ldots + KR_{cell}C_{bl}$$

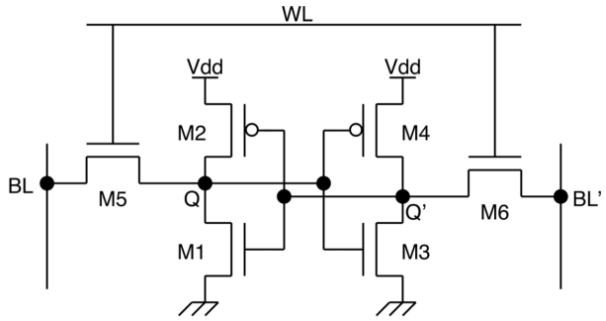
$$\tau_{cell} = R_{cell}C_{bl}(1 + 2 + \ldots + K)$$

$$\tau_{cell} = R_{cell}C_{bl}K(K+1)/2$$



SRAM Cell:





Pre-charge:
1) Phi = 0, WL = 0, and pre charge with VDD/2 instead of VDD.

Read:
1) WL of the target cell = '1' and all other WL = '0'.
2) Read BL - BL'.

Write:
1) All WL = 0.
2) Write your desired value on the bit lines using drive buffers.
3) WL = 1 of the target cell.

The only disadvantage of SRAM memories are their VERY large area. Why?
1) We are using 6 transistors just to make a single SRAM cell. In NAND and NOR ROMs, we had only a single transistor for each cell.
2) Hena fel layout kol SRAM cell me7taga:
   a) Nwell for PMOS transistors.
   b) Ground rail.
   c) VDD rail.

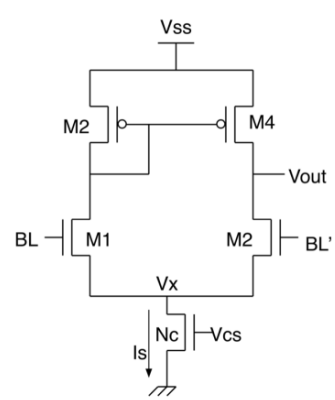
NOR is faster than NAND, but NAND has less area than NOR.

Sense Amplifiers

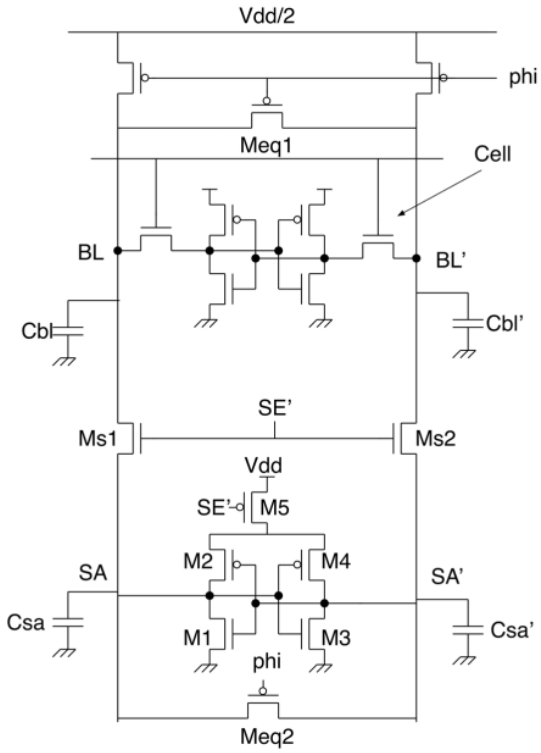
- Sense amplifiers resolve the area-delay dilemma in memories.
- A sense amplifier can be:
  1. Single-Ended 5T OTA.
  2. Latch-Based Amplifier.

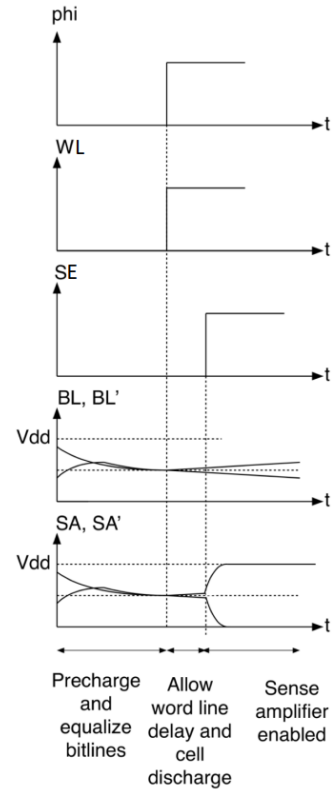
i. Single-Ended 5T OTA sense amplifier:

$$\Delta t = \frac{C_{out}\Delta V_{out}}{I_{available}}$$



ii. Latched-Based Sense Amplifier:





1. M5: da enable transistor, beyosta5dam 3ashan yewasal aw yefsel el supply 3an el latch.
2. Ms1, Ms2: pass transistors, ya2ema yewasalo el cell bel sense amplifier ya2emo yefseloh (SE: Sense Enable).
3. Meq1 & Meq2: mohemet-hom enohom athna2 el pre-charging yedmano en BL & BL' teb2a el voltages beta3et-hom equalized bzbt. Fa homa beye3melo shorting 5elal fatret el pre-charging bs.
4. Delay:

$$\Delta V_{SA} = \Delta V_{SA}(0)\, e^{t/\tau}$$

$$\tau = \frac{C_{SA}}{g_m}$$

$$t = \tau \ln\left(\frac{\Delta V_{SA}}{\Delta V_{SA}(0)}\right)$$

Notice en el fekra el fundamental beta3t el sense amplifiers heya enaha bet5aleek te charge/discharge caps asghar bekteer men beta3t el bit line fa el denya beteb2a asra3, sawa2 fel 5T OTA beteb2a el output capacitance beta3t M2 & M4, aw hena el capacitance Csa.